



系統硬件安全

李琦

清華大學網研院



本章的內容組織



第一節 硬件攻擊技術

- 漏洞攻擊
- 硬件木馬
- 硬件故障
- 側信道攻擊

學習硬件攻擊的基本方法，
掌握系統硬件攻擊原理

知己知彼，百戰不殆，瞭解攻
擊手段才能制定安全策略



第二節 硬件防護技術

- 木馬檢測技術
- 隔離技術
- 密碼技術
- 側信道防護

熟悉硬件防護技術，理解系統
硬件防禦的基本策略

實踐出真知，在實踐中剖析攻擊技
術，進而構建切實可靠的防禦體系



第三節 典型漏洞分析

- MOLES
- Meltdown
- Spectre
- VoltJockey

結合典型漏洞，深入分析硬件
攻擊技術的具體實現



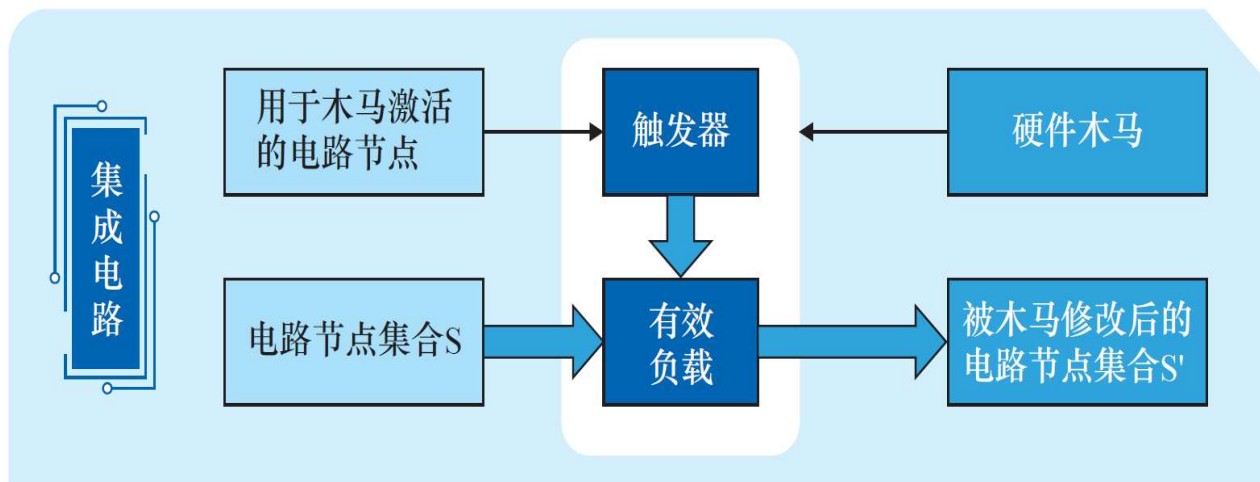
第1節 硬件攻擊技術

- ✓ 漏洞攻擊
- ✓ 硬件木馬
- ✓ 硬件故障
- ✓ 側信道攻擊



威脅二：硬件木馬

硬件木馬是指被第三方故意植入或設計者有意留下的特殊模塊和電路，硬件木馬一般由觸發器和有效負載兩部分組成。這些模塊或電路潛伏在正常硬件中，僅在特定條件下被觸發，實施惡意功能（如竊取敏感信息、拒絕服務等）



不可信的產業鏈

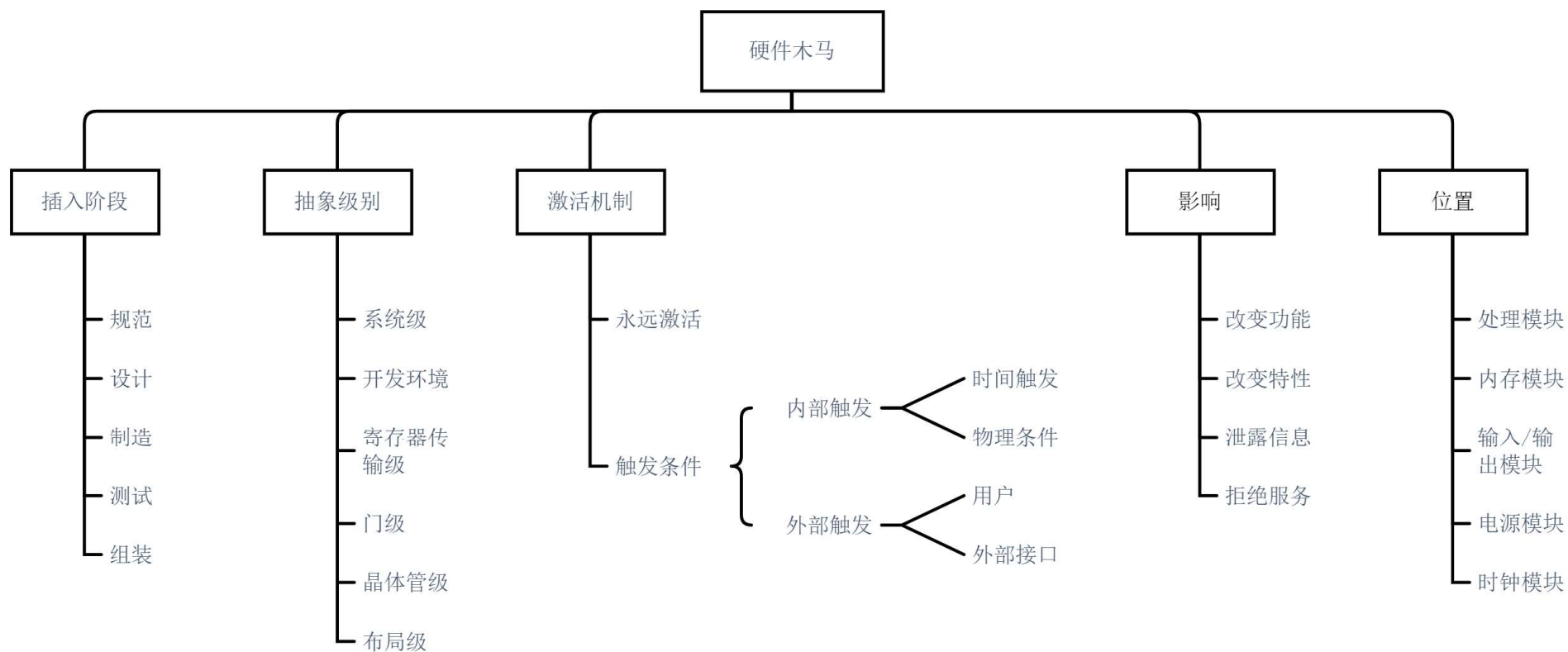


硬件木馬通常由不可信合作廠商引入，目前我國的高端服務器、操作系統等核心軟硬件大部分進口，木馬和後門問題不容樂觀



木馬分類

- 硬件木馬可以基于5個特徵來分類：**插入階段**、**抽象級別**、**激活機制**、**影響和位置**





木馬 ≠ 故障

- **電路故障**是指在電路的生產加工過程中由于工藝或者流程不完美造成的電路缺陷
- **硬件木馬**是指由人爲蓄意加入的指定功能之外的惡意電路模塊。
電路故障通常能够被檢測到而硬件木馬的檢測比較困難

	電路故障	硬件木馬
激活	通常在已知的功能狀態下	任意電路中間節點特定狀態的組合或者序列 (可以爲數字或者模擬信號)
植入	偶然 (製造過程的某些缺陷)	蓄意 (由產業鏈的攻擊者植入)
作用	電路功能或參數錯誤	電路功能參數錯誤、洩漏信息等



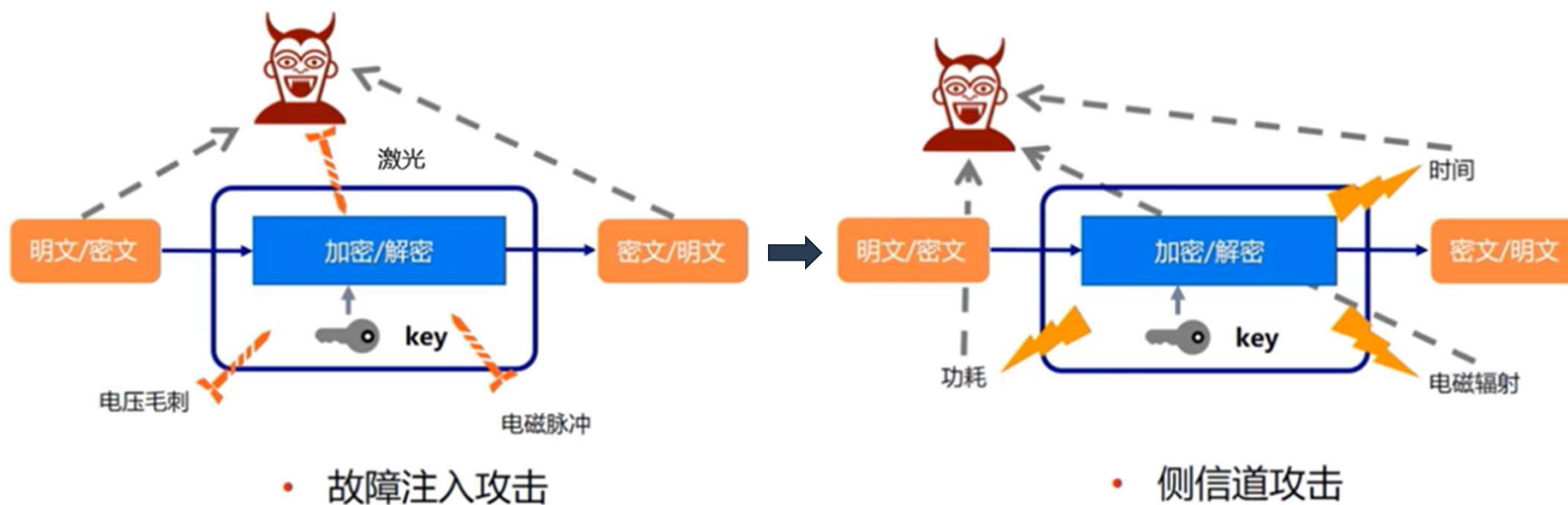
軟件木馬與硬件木馬

- **軟件木馬**是一種帶有惡意代碼的軟件，與硬件木馬攻擊目標相似，軟件木馬通常可以在應用領域內解決，而硬件木馬一旦植入就很難移除

	軟件木馬	硬件木馬
激活	存在惡意代碼的軟件，在代碼運行過程中激活	存在集成硬件中，在硬件運行過程中激活
感染	用戶交互過程中傳播	由集成硬件設計流程中不可信的參與方植入
補救措施	通過軟件更新移除	流片一旦加工成型就不能移除



威脅四：旁路側信道



側信道攻擊：利用硬件系統的旁路信息實施攻擊

側信道——不接觸攻擊



硬件隔離能保證安全嗎

安全吗
真的安全吗
你确定安全吗
你真的确定安全吗

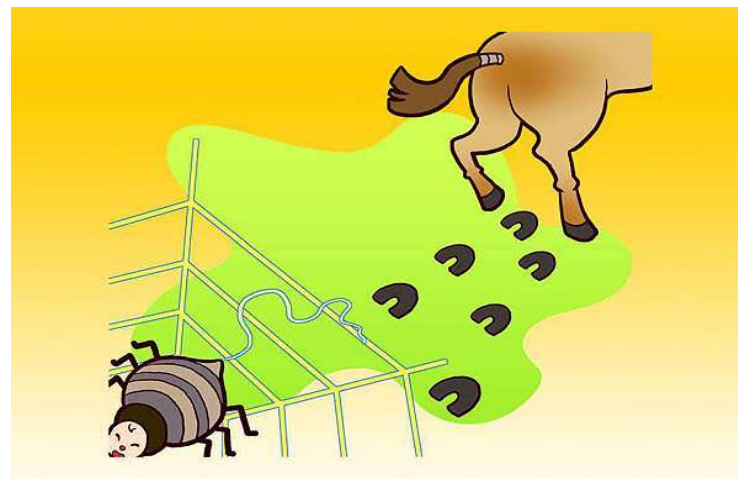
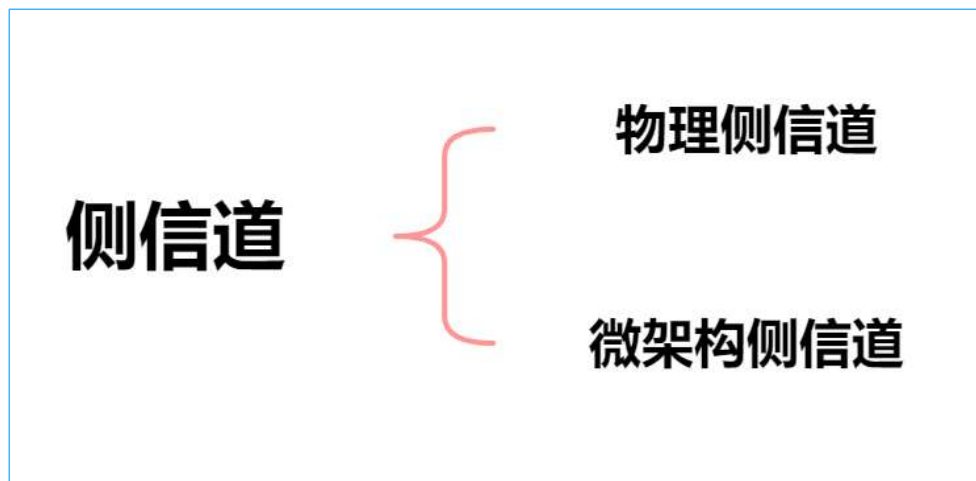


側信道攻擊



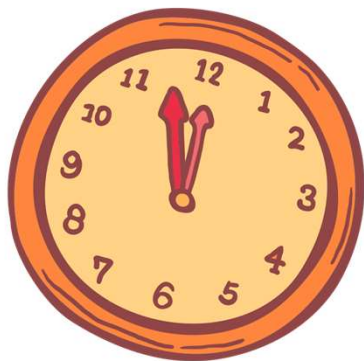
側信道攻擊

- **側信道攻擊** (SCA, Side Channel Attack) : 基于從密碼系統的物理實現中獲取的信息, 例如時間信息、功率消耗、緩存使用等
- 側信道攻擊需要的**設備成本低**、**攻擊效果顯著**, 嚴重威脅了密碼設備的安全性

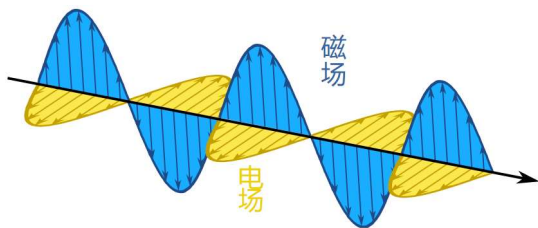




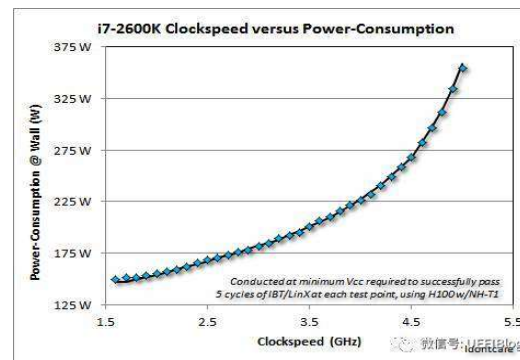
物理側信道



時間側信道



電磁側信道



功耗側信道



聲波側信道



時間側信道示意

時間側信道通過系統在不同行為流程下所表現出的執行時間差異，推斷有用的信息，以達成或輔助攻擊

- 假如一個網站使用如下代碼

```
$username = $this->input->post("username")
$password = $this->input->post("password")
if ($this->is_valid_user($username) == false) {
    echo "Invalid login";
    return;
}
if ($this->verify_password($username, password) == false) {
    echo "Invalid login";
    return;
}

echo "Hello, " . $username;
```

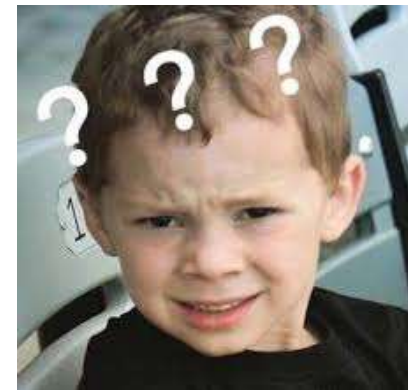
該段代碼的邏輯如下：

1. 若用戶名不存在，則登錄失敗
2. 若用戶名存在但密碼不正確，則登錄失敗
3. 只有當用戶名和密碼都正確，才返回成功



時間側信道攻擊

- 以上代碼的邏輯沒有問題，并且兩次錯誤的返回的字符串信息一致，避免了攻擊者根據該信息判斷出用戶名是否存在，而進行後續的針對性的攻擊
- **真的沒有問題了嗎？** 考慮下面的攻擊方式：
 - ① 攻擊者從本地提交post請求，其中，變化username，而保持password為任意固定值，例如 “666”
 - ② 攻擊者統計每次從發出請求起到收到服務器回執所耗費的時間，對於同一組用戶名和密碼，也可以通過測量多次取平均值、中值等方法消除網絡延遲引入的時間抖動
 - ③ 對於統計時間較長的請求，認為其所對應的用戶名是存在的
 - ④ 基于存在的用戶名，進一步發動針對性的攻擊





功耗側信道攻擊

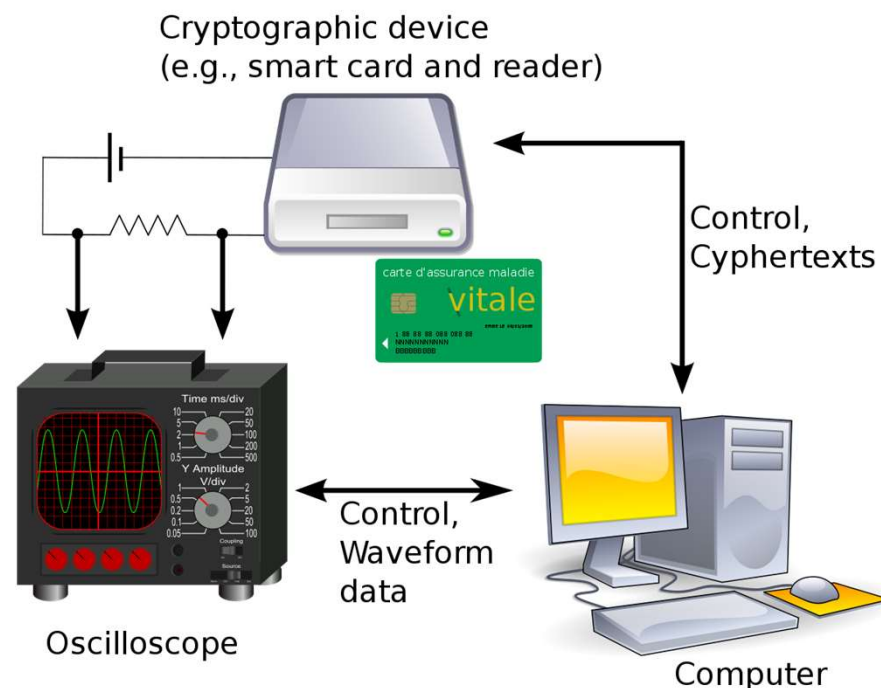
- 功耗側信道分析最早是由Paul Kocher等提出
- CPU 執行不同指令的時候會有不同的功耗特徵：
不同的指令觸發的半導體數量不同，有的指令還會訪問內存、緩存等等，各種各樣的因素會導致不同的指令執行的時候會產生不同的功耗特徵
- 利用不同功耗特徵可以推測出CPU執行過程中的某些信息

功耗側信道攻擊可分為以下三種：

簡單功耗分析

差分功耗分析

相關功耗分析





簡單功耗分析

- **簡單功耗分析 (simple power analysis, SPA)**：攻擊者通過直接分析設備隨時間的功耗曲線，以及曲線特徵和分析者經驗可直觀得出CPU執行的順序或指令

受害者芯片

1. 從串口讀取一行用戶輸入的密碼
2. 跟固件中一個寫死的密碼 h0px3 進行比較
3. 向串口輸出匹配結果

攻擊者

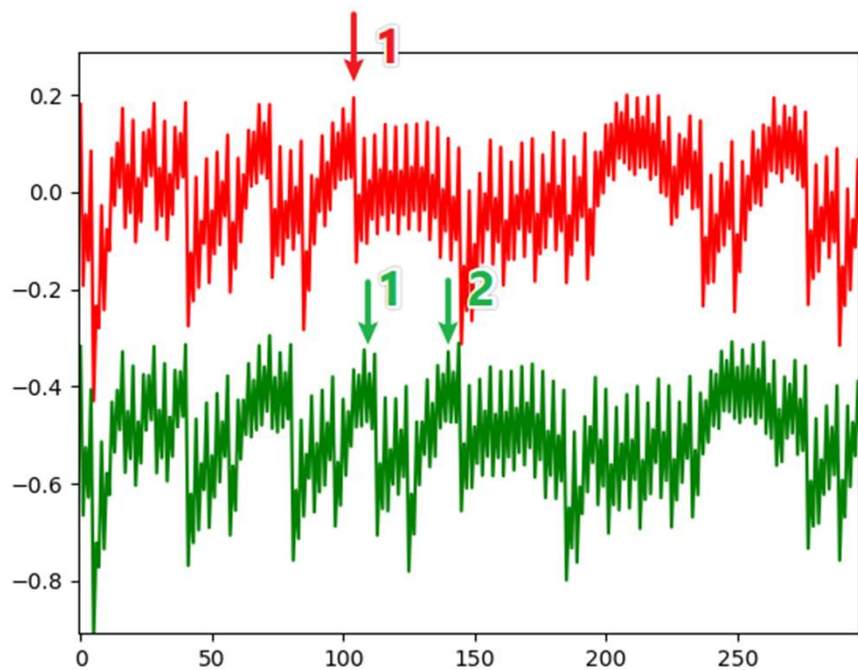
通過硬件接口監測受害者芯片電壓（功耗）變化情況，依次輸入兩個字符串 “a\n” 和 “h\n”



觀察兩次功耗曲線的差異

簡單功耗分析示意

兩次功耗曲綫不一樣？



正確密碼：h0px3

a

h

- 一次字符對比 $a \neq h$ ，就退出循環



- 兩次字符對比，第一次 $h=h$ ；第二次輸入不是0，退出循環



密碼爆破

密碼位數：5位

密碼字符：36個（字母+數字）

爆破複雜度： $O(36^5)$



功耗分析

什么叫降维打击



密碼位數：5個1位

密碼字符：36個（字母+數字）

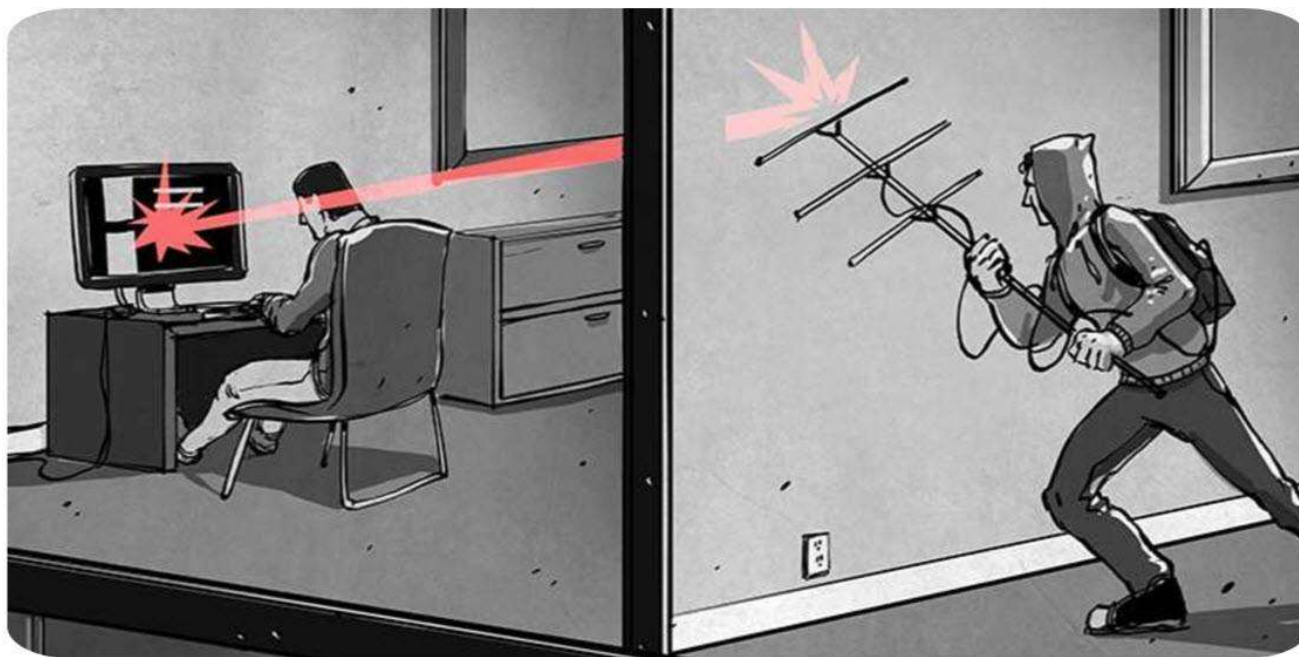
爆破複雜度： $O(36)$





電磁側信道

- 無論是JavaScript亦或是C語言編寫的甚至定制硬件電路也適用
- 任何程序都要靠硬件電路來運行實現，由此，也帶來一些有趣的副作用：運動電荷激發磁場——詹姆斯·克拉克·馬克斯韋爾



利用電磁場竊取信息

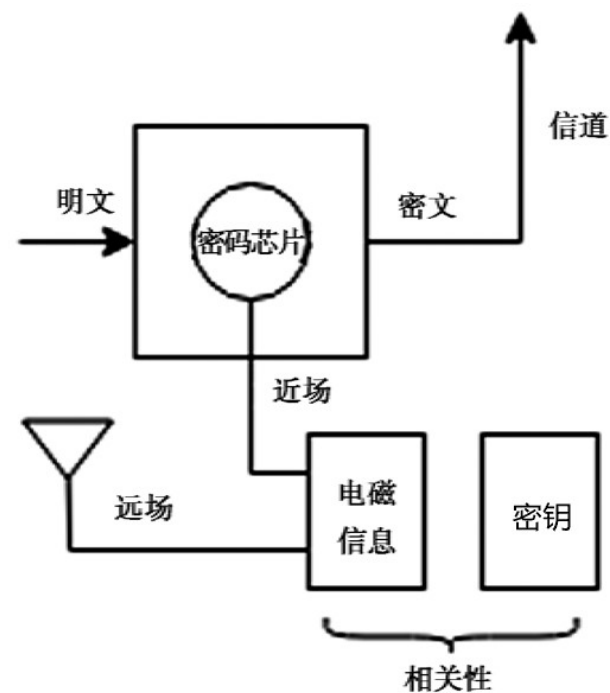


電磁側信道攻擊

- **電磁分析 (electromagnetic analysis) 攻擊**：由比利時學者Quisquater和Samyde于2000年首先提出，與數據相關的電流在處理器中的變化可以導致磁場的變化，攻擊者分析磁場進而分析出數據，電磁分析與功耗分析方法類似——**非接觸式功耗分析**

$$d\vec{B} = \frac{\mu I d\vec{l} \times \vec{r}}{4\pi r^2}, \vec{B} = \int d\vec{B}$$

- 密碼芯片在工作過程中會**不可避免**的對外產生**電磁輻射**，輻射的信息和芯片內部的數據存在一定的**相關性**





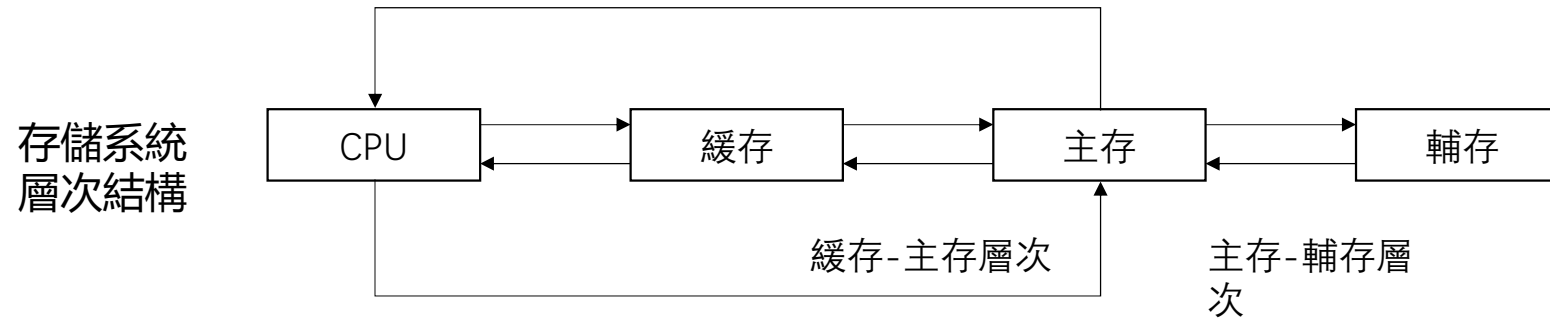
微架構側信道

- 現代處理器的優化技術，包括亂序執行和推測機制等，對性能至關重要
- 隨著處理器性能的不斷提升，與之相關的漏洞也越來越多
- 以Meltdown和Spectre為代表的側信道攻擊表明：由于異常延遲處理和推測錯誤而執行的指令結果雖然在架構級別上未顯示，但仍可能在處理器**微架構狀態**中留下痕迹，通過隱蔽信道可將微架構狀態的變化傳輸到架構層，進而恢復出秘密數據





Cache



Cache組成

Cache存儲體

存放由主存調入的指令與數據塊

地址轉換部件

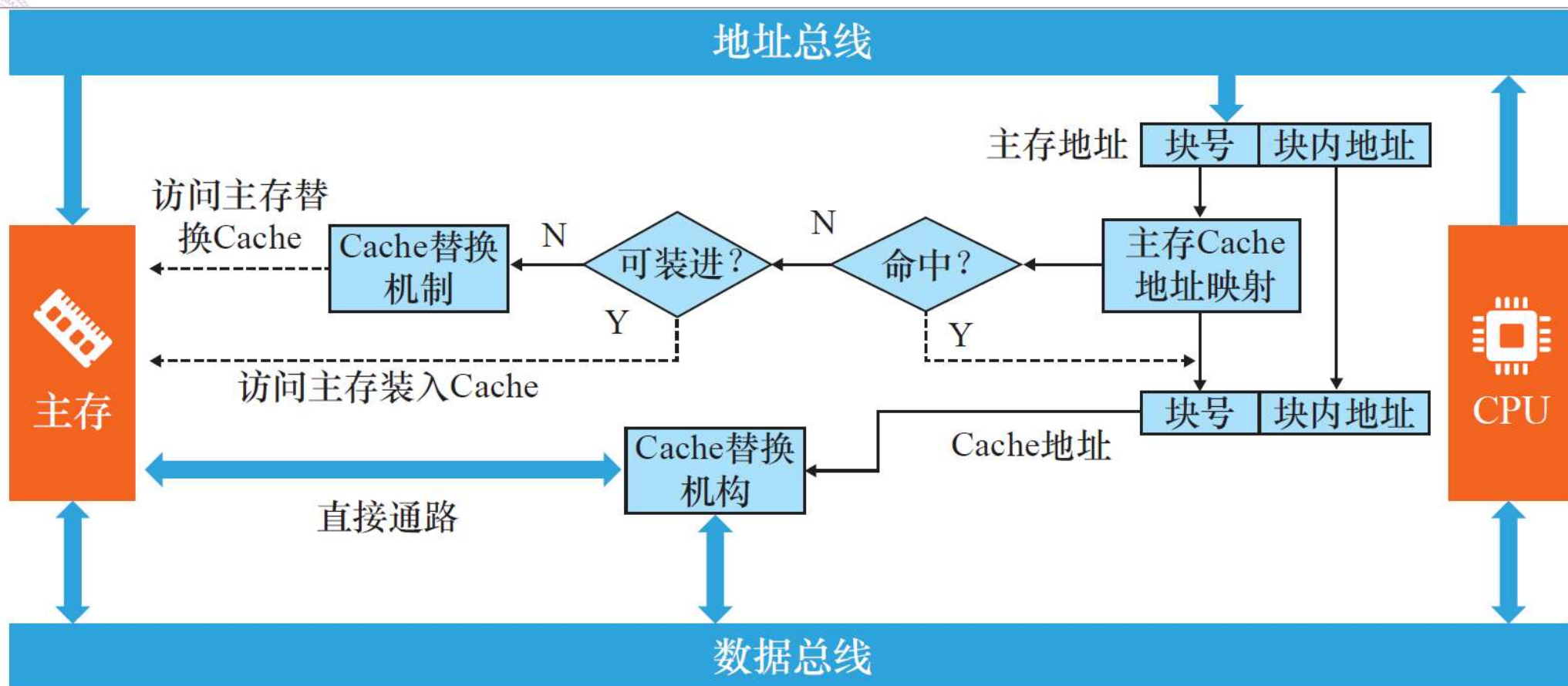
建立目錄表以實現主存地址到緩存地址的轉換

替換部件

在緩存已滿時按一定策略進行數據塊替換，并修改地址轉換部件



Cache工作原理



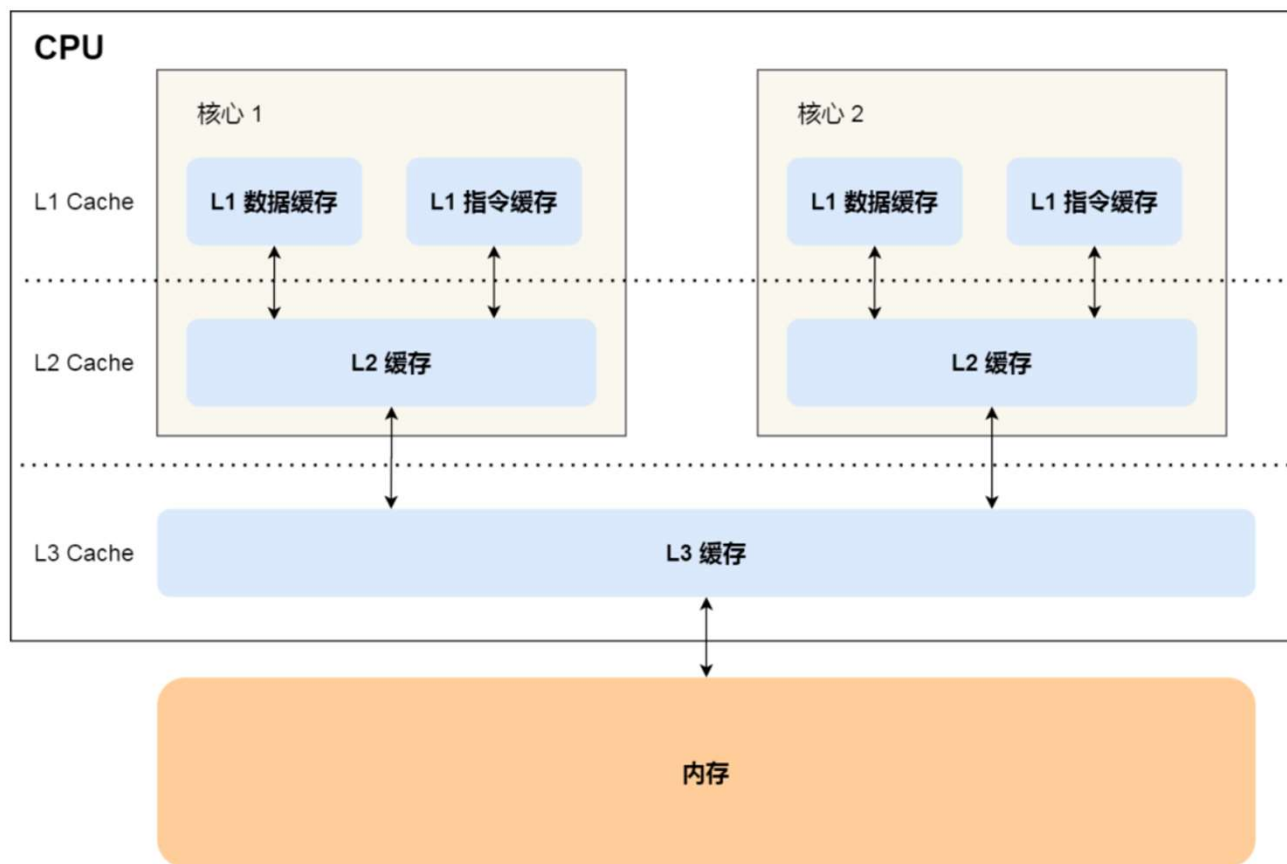
Cache基本結構



Cache結構

部件	CPU 访问所需时间
L1 Cache	2~4 个时钟周期
L2 Cache	10~20 个时钟周期
L3 Cache	20~60 个时钟周期
内存	200~300 个时钟周期

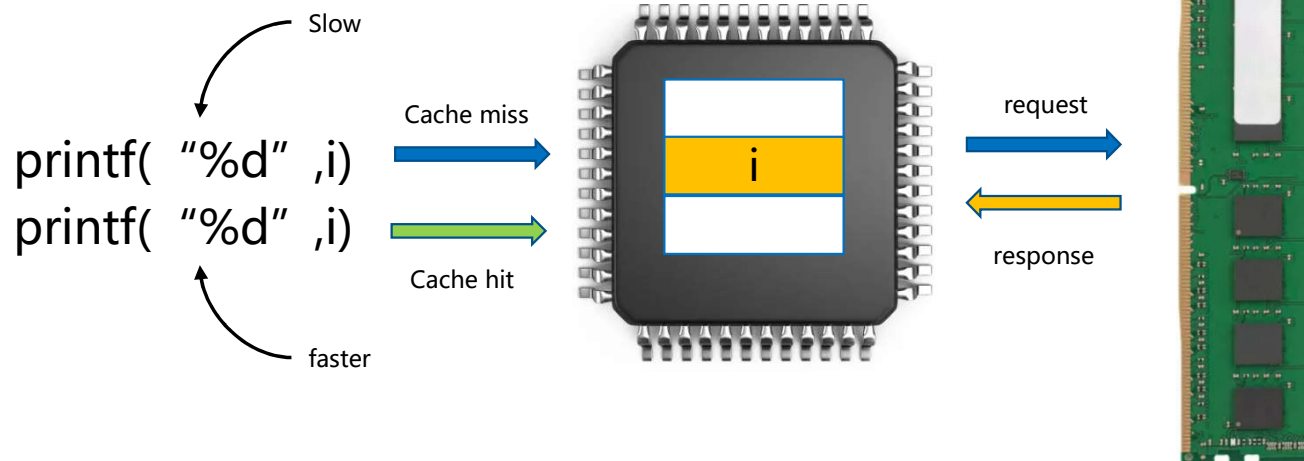
- 數據與指令分離
- L1、L2單核獨有
- L3多核共享





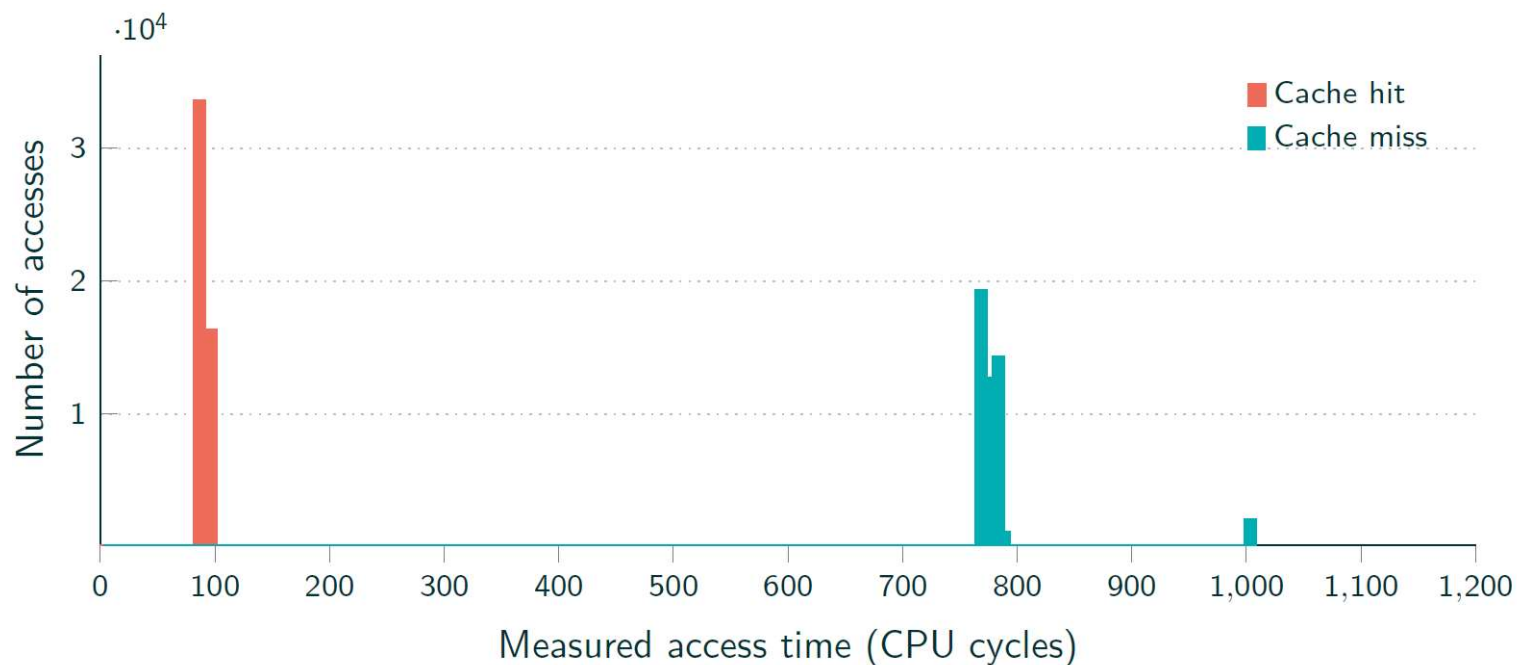
Cache側信道

攻擊原理：多核之間**cache數據共享**，而cache命中和失效對應**響應時間有差別**，攻擊者可以通過訪問時間的差異，推測cache中的信息，從而獲得隱私數據





時延分析



根據時延分析構造緩存側信道，目前，Cache側信道攻擊主要包含四種方法：

Prime-Probe

Flush-Reload

Evict-Reload

Flush-Flush



Prime-Probe

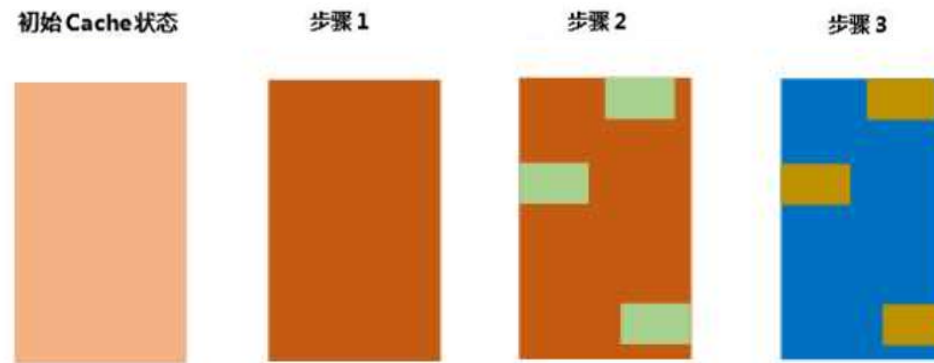
步驟1. Prime: 攻擊者用預先準備的數據填充特定多個cache組

步驟2. Trigger: 等待目標進程響應服務請求，將cache數據更新

步驟3. Probe: 重新讀取Prime階段填充的數據，測量并記錄各個cache組讀取時間



針對cache進行操作，先填





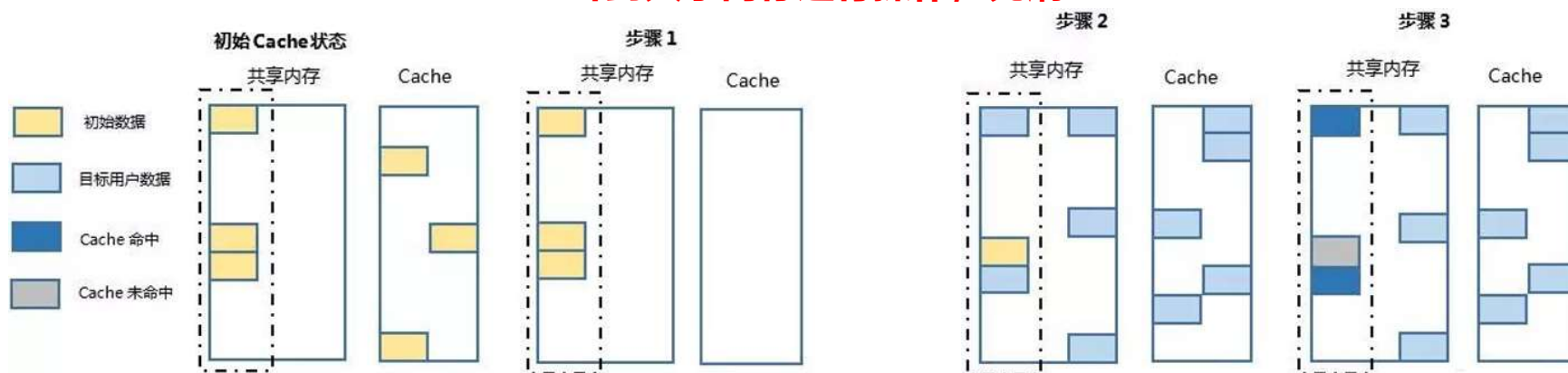
Flush-Reload

步驟1. Flush: 將共享內存中特定位置映射的cache數據驅逐

步驟2. Trigger: 等待目標進程響應服務請求, 更新cache

步驟3. Reload: 重新加載Flush階段驅逐的內存塊, 測量并記錄cache組的重載時間

針對共享內存進行操作, 先刷



Evict+Reload和Flush+Reload的攻擊流程基本一致, 通過驅逐 (eviction) 的方式, 用于替代 Flush+Reload中的Flush指令缺失的情況

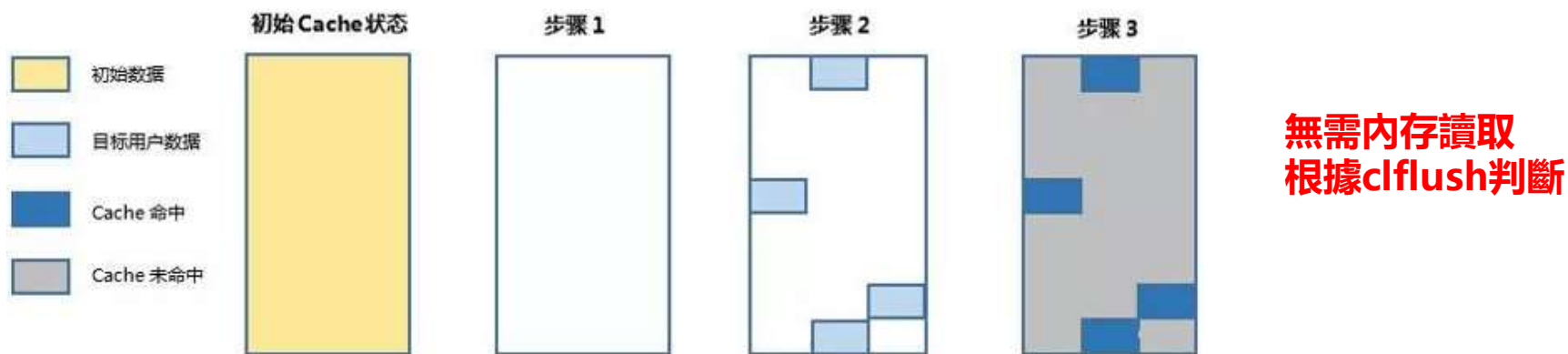


Flush-Flush

步驟1：通過flush清空cache原始數據

步驟2：等待目標進程運行，更新cache，并刷新共享緩存行，測量刷新時間

步驟3：根據測量時間判斷原始數據是否被緩存



Flush-Flush攻擊是基于cflush指令執行時間的長短來實施攻擊的，如果數據沒在Cache中則cflush指令執行時間會比較短，反之若有數據在cache中則執行時間會比較長，與其它Cache攻擊不同，Flush Flush側信道攻擊技術在整個攻擊過程中是不需要對內存進行存取的，因此該攻擊技術更加隱蔽



Cache側信道小結

	Prime-Probe [1]	Flush-Reload [2]	Flush-Flush [3]
驅逐緩存集方式	利用緩存結構中的衝突，從緩存中驅逐目標地址	利用cflush指令刷新指定內存位置，使其從緩存中驅逐	利用cflush指令刷新指定內存位置，使其從緩存中驅逐
判斷依據	重新讀取填充數據，讀取速度慢的位置，其對應的cache被目標程序使用過	重新讀取刷新的內存，讀取速度快的位置，是被目標程序使用過的	再次使用cflush指令刷新指定內存位置，cflush指令運行時間長的說明被目標程序使用過
攻擊特點	無需共享內存，需要訪問內存數據	共享內存，需要訪問內存數據	共享內存，無需訪問內存數據

Cache側信道攻擊通過構建細粒度、高隱蔽性的Cache側信道，打破設備的隔離保護機制，從受害者內存中（內存數據一般是明文）提取**加密密鑰**等敏感數據，對系統硬件安全造成了嚴重威脅

[1] Osvik D A, et al. Cache attacks and countermeasures: the case of AES. Springer Berlin Heidelberg, 2006.
[2] Yarom, et al. FLUSH+ RELOAD: A high resolution, low noise, L3 cache side-channel attack. Security 2014.
[3] Gruss D, et al. Flush+ Flush: a fast and stealthy cache attack. Springer International Publishing, 2016.



第2節 硬件防護技術

- ✓ 木馬檢測技術
- ✓ 隔離技術
- ✓ 密碼技術
- ✓ 側信道防護



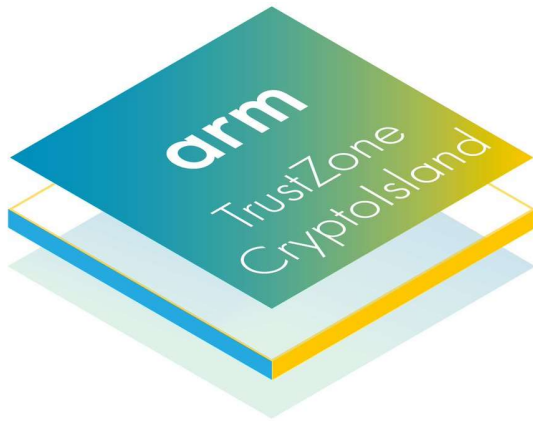
硬件防護技術





防護二：隔離技術

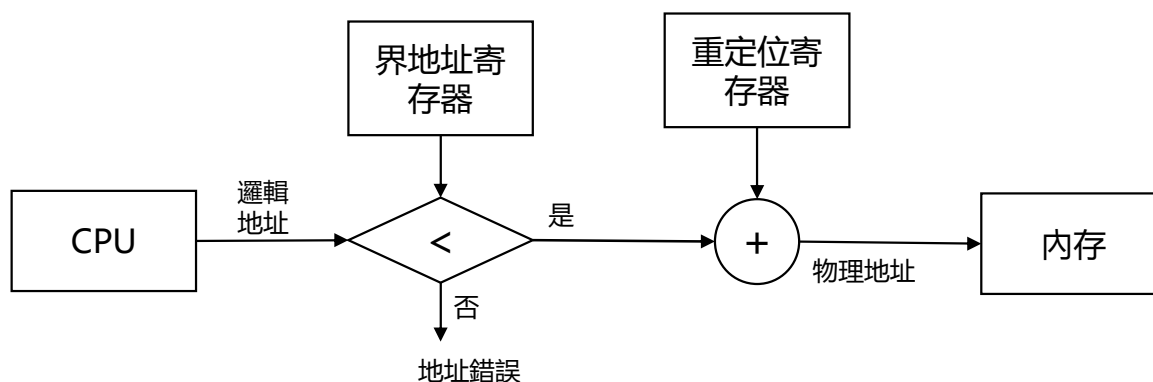
- **隔離技術**是一種訪問控制手段，用于限制用戶訪問非授權的計算資源。借助硬件隔離技術可以實現一些計算資源的共享（存儲器隔離）、安全計算環境和不安全計算環境之間的隔離（TrustZone和SGX）





存儲器隔離

- 存儲器隔離是現代處理器通常具有的一種特性
- 實現手段：存儲器保護技術、EPT硬件虛擬化技術和存儲加密技術

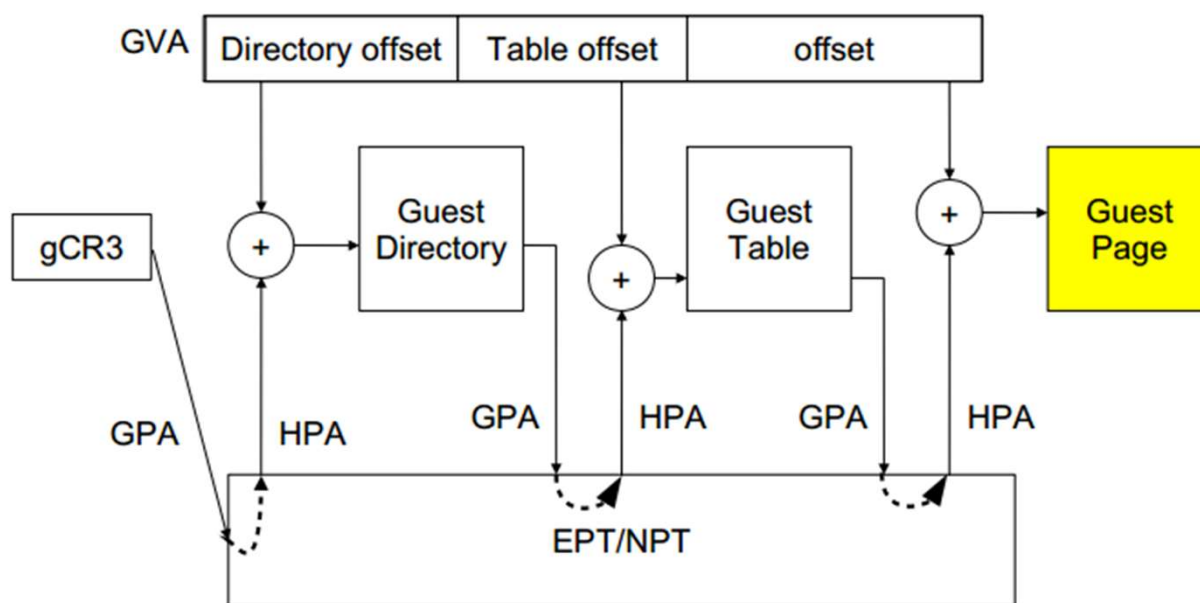


- 系統使用**IOMMU** (IO Memory Management Unit) 限制設備能夠訪問的存儲器
- 通過**重定位寄存器**和**界地址寄存器**實現IOMMU，重定位寄存器包含物理地址，界寄存器包含邏輯地址



EPT硬件虛擬化技術

EPT是Intel針對軟件虛擬化性能問題推出的硬件輔助虛擬化技術



- **GVA:** Guest 虛擬地址
- **GPA:** Guest 物理地址
- **HVA:** Host 虛擬地址
- **HPA:** Host 物理地址
- **gCR3:** Guest CR3



EPT地址轉換

Future Extensions: EPT

EPT: Overview



- Intel® 64 page tables
 - Map **guest-linear** to **guest-physical** (translated again)
 - Can be read and written by guest
- New EPT page tables under VMM control
 - Map **guest-physical** to **host-physical** (accesses memory)
 - Referenced by new **EPT base pointer**
- No VM exits due to **page faults**, **INVLPG**, or **CR3** accesses



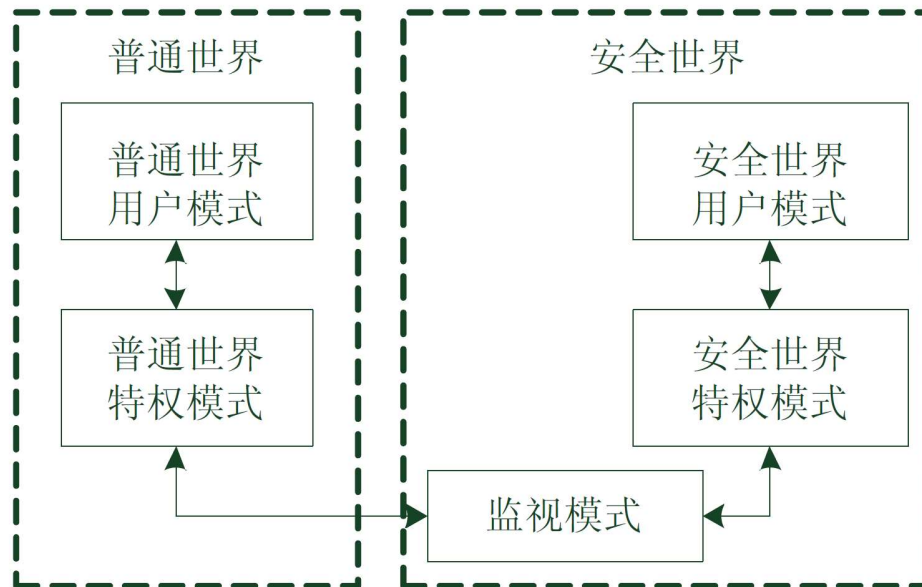
8

- **CR3**將客戶機程序所見的**客戶機虛擬地址**（GVA）轉化為**客戶機物理地址**（GPA），然後在通過**EPT**將**客戶機物理地址**（GPA）轉化為**宿主機物理地址**（HPA）。這兩次轉換地址轉換都是由CPU硬件自動完成，其轉換效率非常高



ARM TrustZone

TrustZone將CPU內核隔離成**安全**和**普通**兩個區域，即單個的物理處理器包含了兩個虛擬處理器核：安全處理器核和普通處理器核。從而單個處理器內核能夠以時間片的方式安全有效的同時從普通區域和安全區域執行代碼



- **非安全核**只能訪問普通世界的系統資源, 而**安全核**能訪問所有資源
- **安全核**與**非安全核**之間的切換通過使用**SMC (Secure Monitor Call) 指令**或者通過硬件異常機制的一個子集實現



ARM TrustZone

安全世界執行

- 如指紋識別
- 密碼處理
- 數據加解密
- 安全認證
-

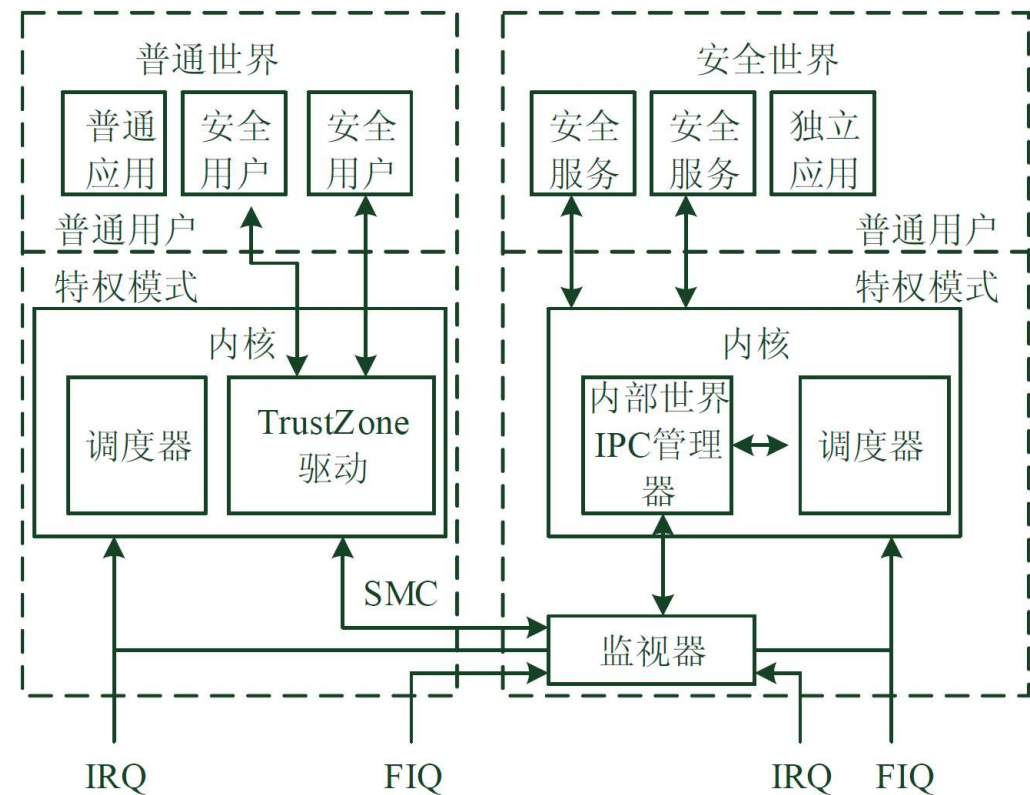
普通世界執行

- 用戶操作系統
- 應用程式
-

普通世界

監視器

安全世界

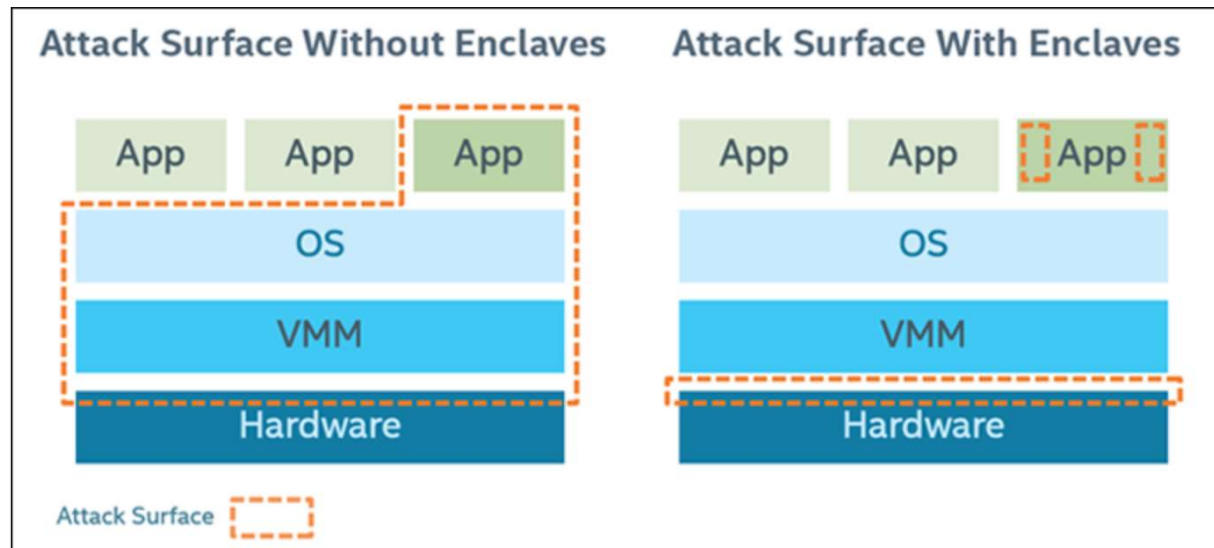


- IRQ: 普通世界的中斷源
- FIQ: 安全世界的中斷源



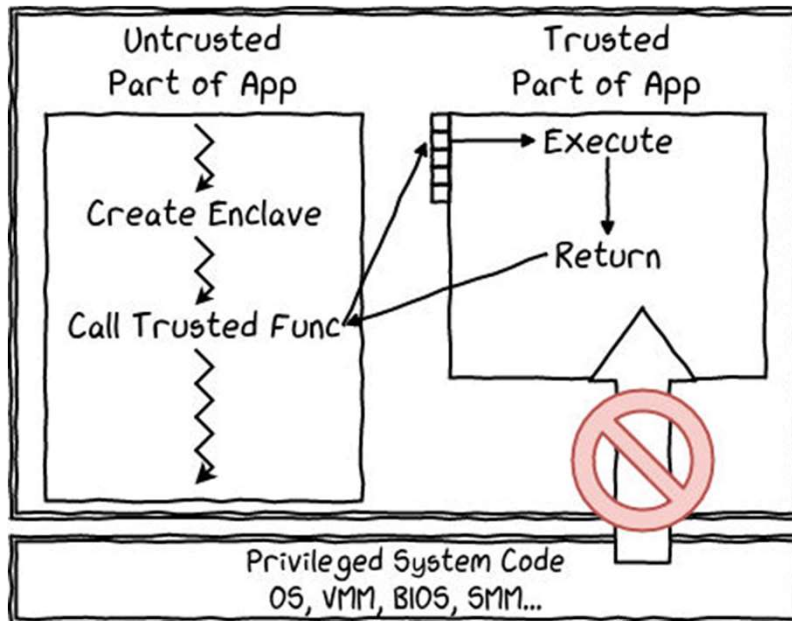
Intel SGX

- **Intel SGX**是一組CPU指令，可讓應用程序創建安全區：應用程序地址空間中的受保護區域，即使存在特權惡意軟件，也可提供機密性和完整性
- 英特爾SGX可以減少應用程序的攻擊面





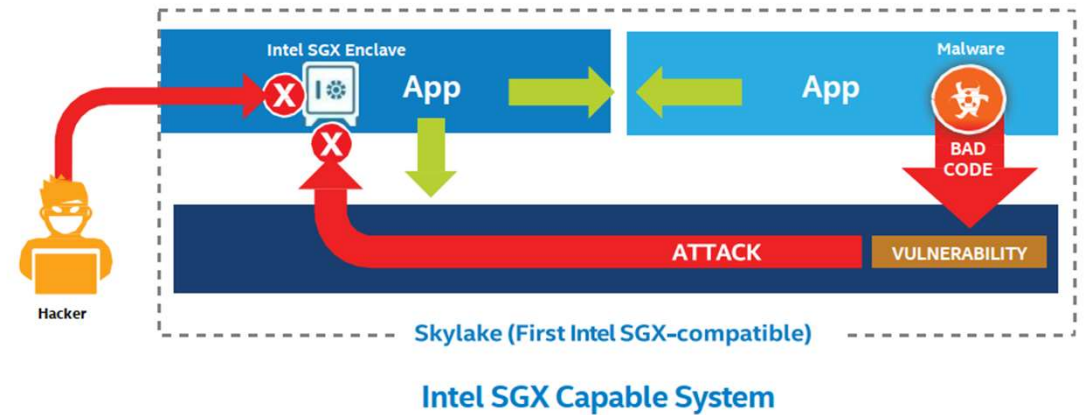
SGX概述



- 應用分為安全和非安全
- Enclave放置在受保護的內存中

SGX特點

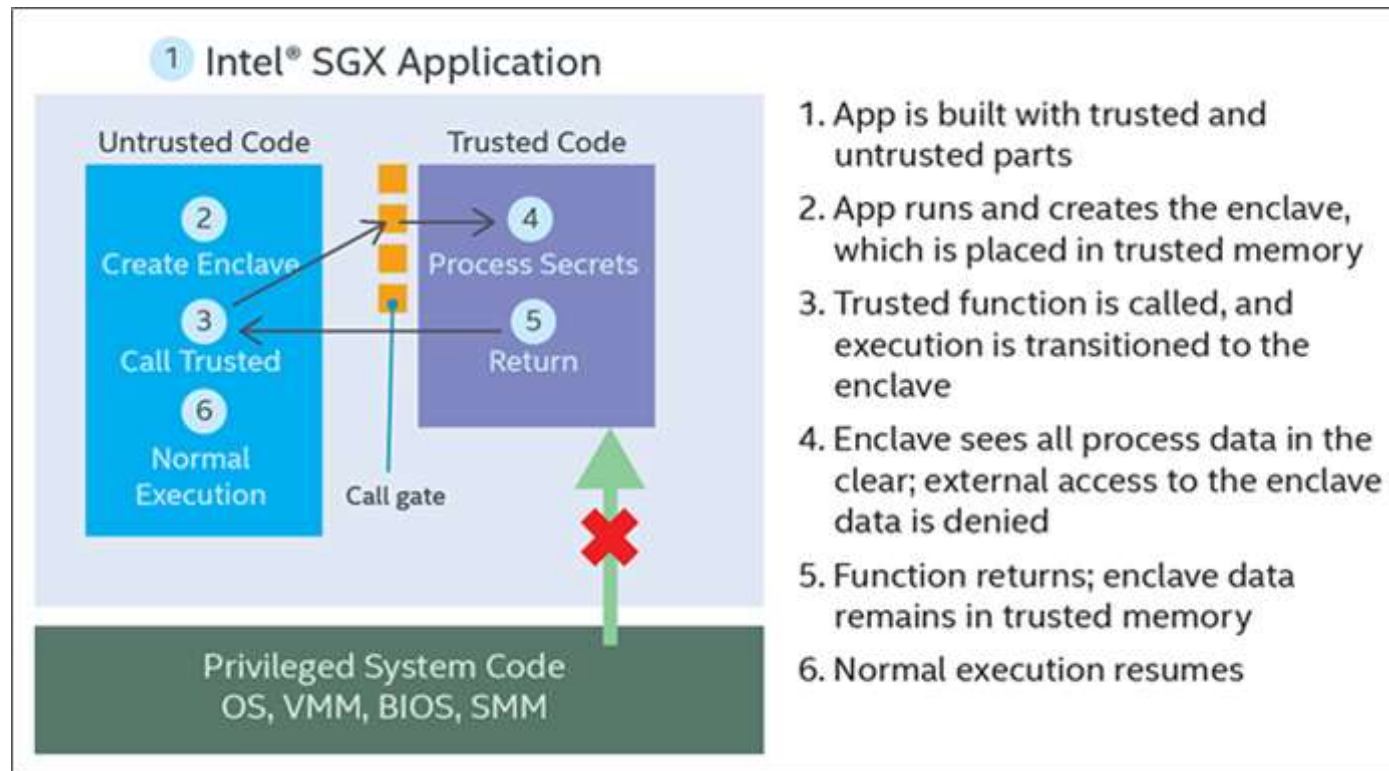
- 機密性和完整性
- 低學習曲綫
- 遠程認證
- 縮小攻擊面





SGX應用分類

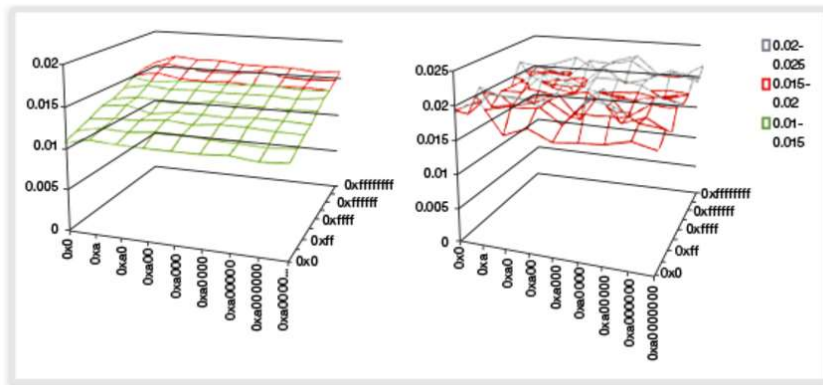
- 使用英特爾SGX進行應用程序設計時，需要將應用程序分為兩個組件:**受信任的組件**和**不受信任的組件**



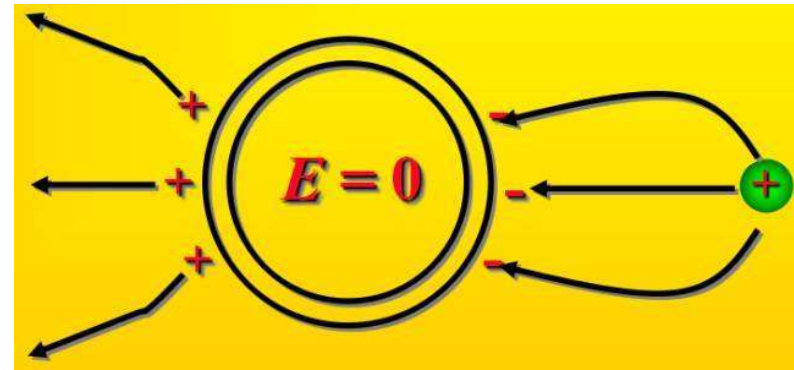


防護四：旁路信息隱藏

- 攻擊者利用旁路信道的輸出來獲得足够的信息，以確定和芯片操作相關的敏感數據。所以他們尋求從低信噪比的旁路信道中恢復信號（試圖**提高信噪比**），因此可以通過**增大噪聲**或者**减小信號**來**降低信噪比**，增加攻擊難度



增加噪聲

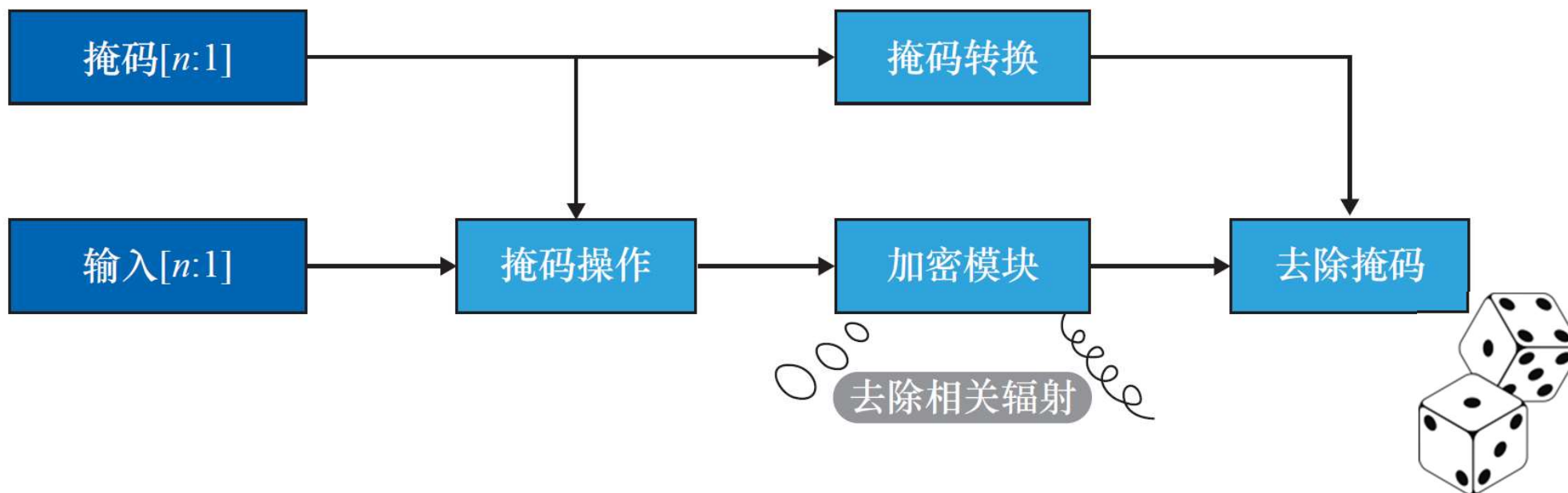


物理屏蔽



掩碼技術

掩碼技術是一種從功能模塊的**中間節點**入手，移除輸入數據與旁路信道相關性的措施，該技術可基于門或者基于模塊來實現

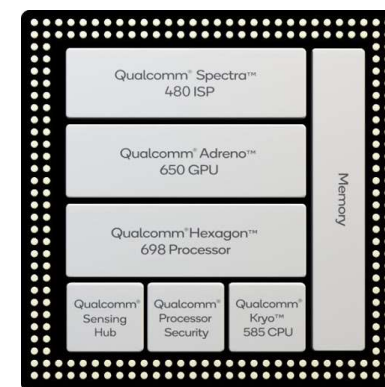




模塊劃分及物理安全

模塊劃分

一些通道會洩漏信息是因為敏感信號被耦合到了其他節點上面，爲了減少這種情況，設計人員可以將芯片操作中的明文操作區和密文操作區分開。除了將芯片不同區域進行物理隔離，還需要將片內共享的基礎設施進行分離，包括供電基礎設施、時鐘基礎設施、和測試基礎設施



物理防護

爲了感知高信噪比的旁路信息，攻擊者需要對受害設備進行長時間的物理訪問，甚至破壞設備。因此拒絕接近、拒絕訪問和拒絕受控是降低攻擊者掛載到旁路信道進行攻擊的關鍵



第3節 典型漏洞分析

- ✓ MOLES
- ✓ Meltdown
- ✓ Spectre
- ✓ VoltJockey



漏洞二：Meltdown概述



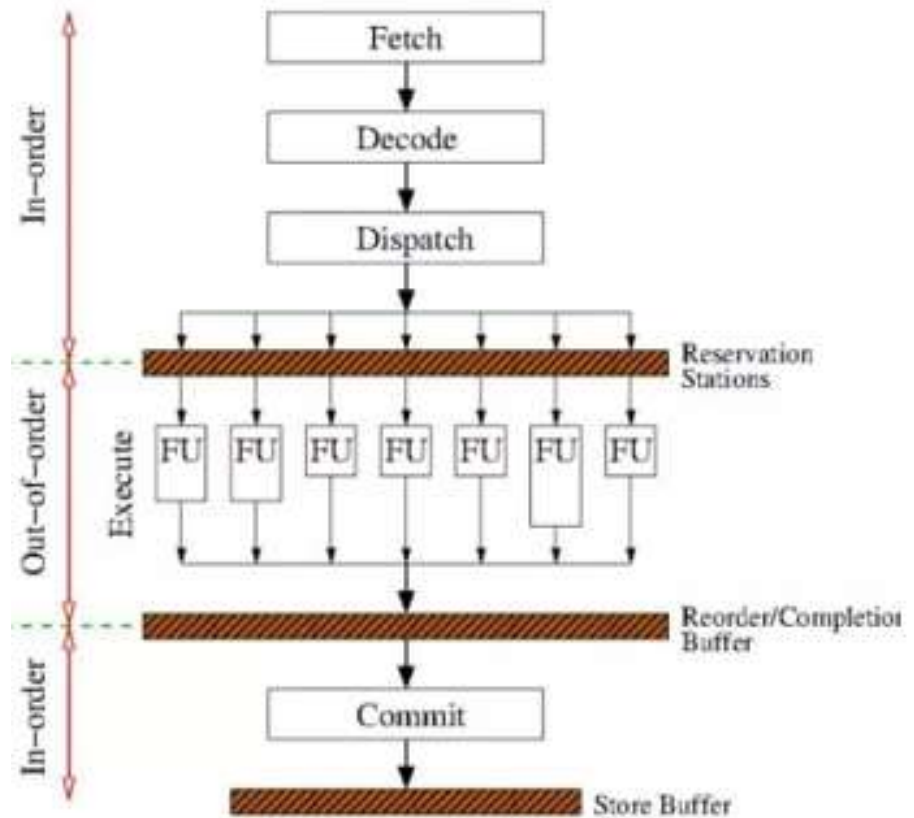
1. 使用亂序執行，攻擊者可以在任何地址讀取數據
2. 洩漏內核內存
3. 影響幾乎所有的Intel處理器和ARM Cortex-A75



亂序執行

亂序執行過程

1. 獲取指令，解碼後存放到執行緩衝區（保留站）
2. 亂序執行指令，結果保存在一個結果序列中
3. 重排，重新排列結果序列及安全檢查（如地址訪問的權限檢查），提交結果到寄存器





攻擊過程

```
1 rcx = kernel address,  
  rbx = probe array  
2 xor rax, rax  
3 mov al, byte [rcx] //step 1  
4 shl rax, 0xc //rax*4096  
5 mov rbx, qword [rbx + rax]
```

目標地址

將數據映射到
緩存頁 (4KB)

在攻擊者緩存
集上留下痕迹



攻擊場景

攻擊者：用戶級特權

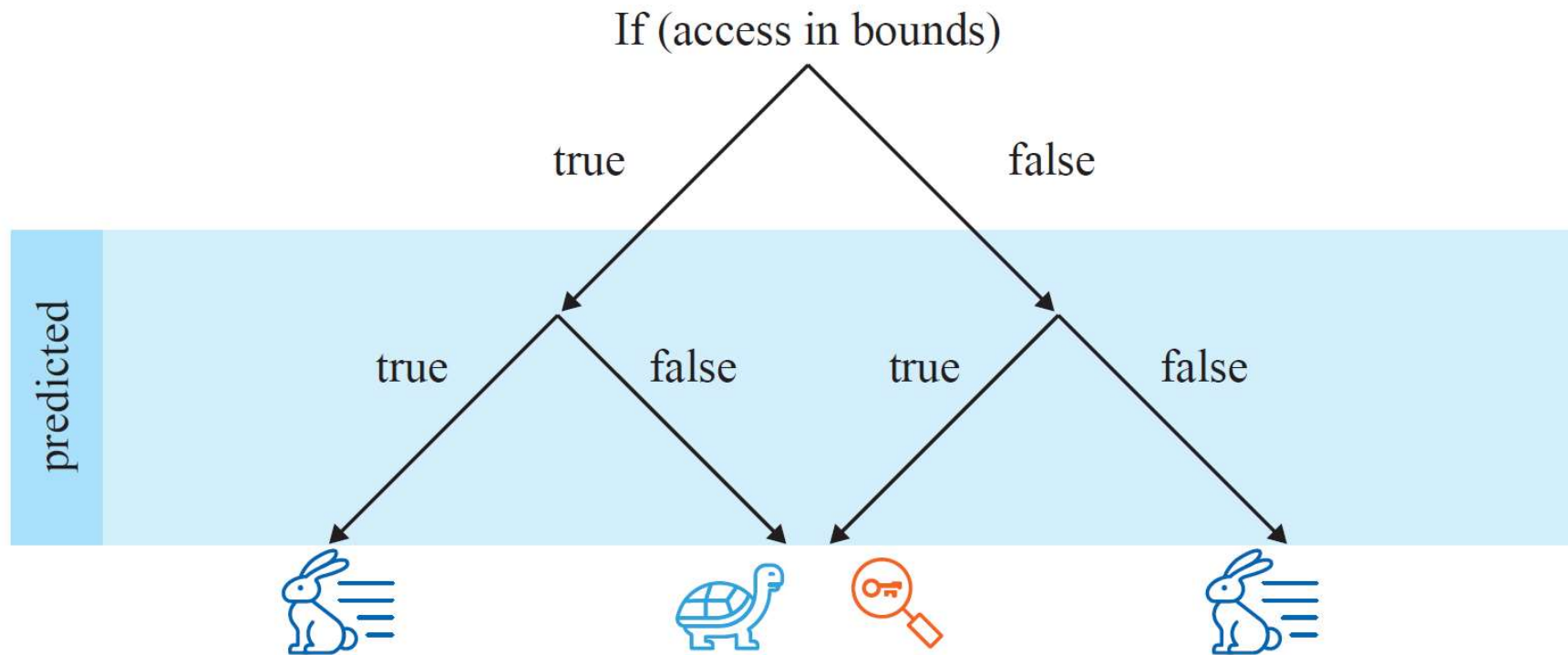
目標：讀取內核數據

- line3、4、5亂序執行
- 敏感數據泄漏到cache
- 安全檢測丟棄違例結果
- Cache側信道提取數據



漏洞三：Spectre

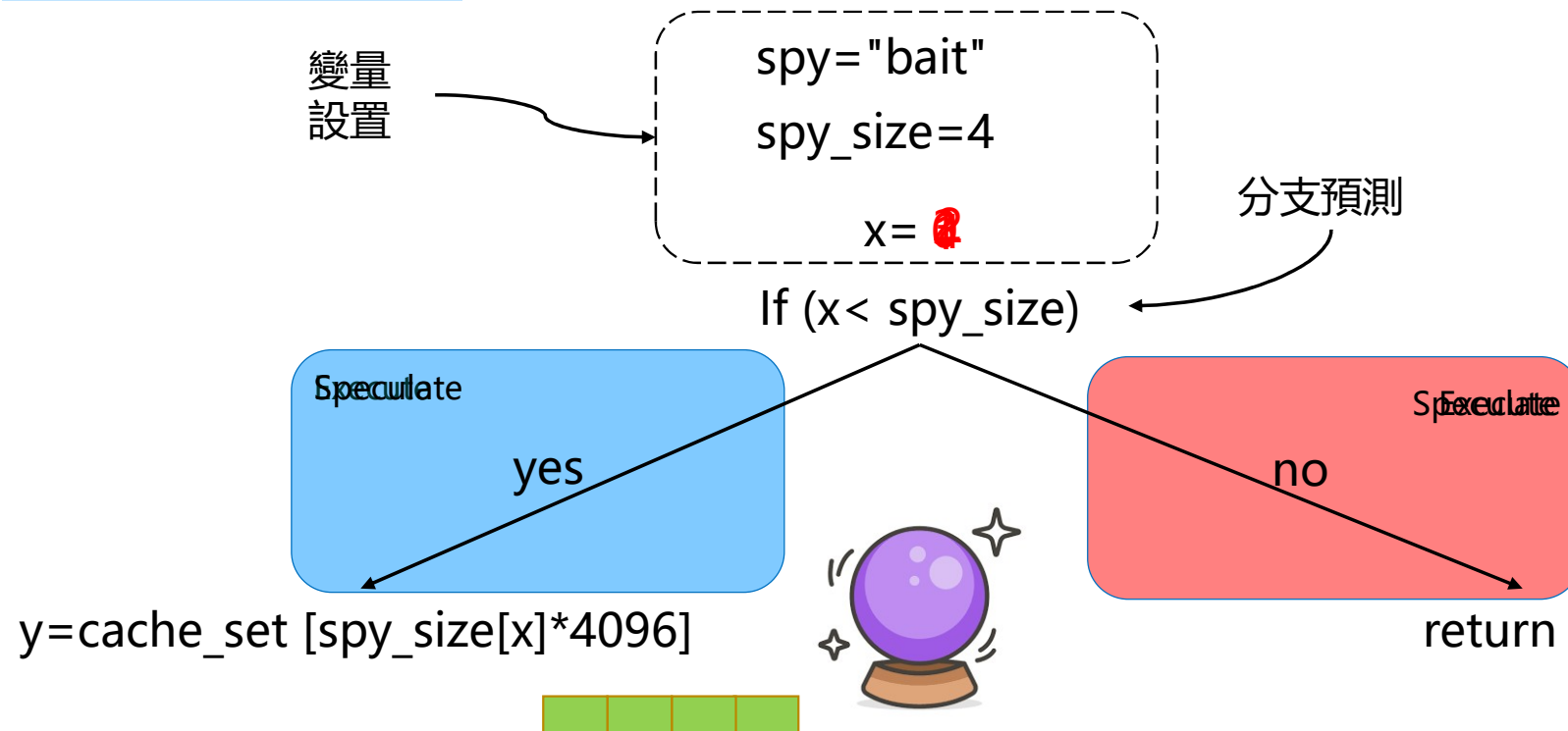
分支預測起在分支指令執行結束之前**猜測**哪一路分支將會被運行，以提高處理器的指令流水綫的性能





攻擊過程

訓練分組預測器





關鍵步驟

與Meltdown類似，Spectre的原理是，當CPU發現分支預測錯誤時會丟棄分支執行的結果，恢復CPU的狀態，但是不會恢復**CPU Cache**的狀態，利用這一點可以突破進程間的訪問限制，從而獲取其他進程的數據

```
if (x < array1_size) { y = array2[array1[x] *  
4096];  
// do something detectable when  
// speculatively executed }
```



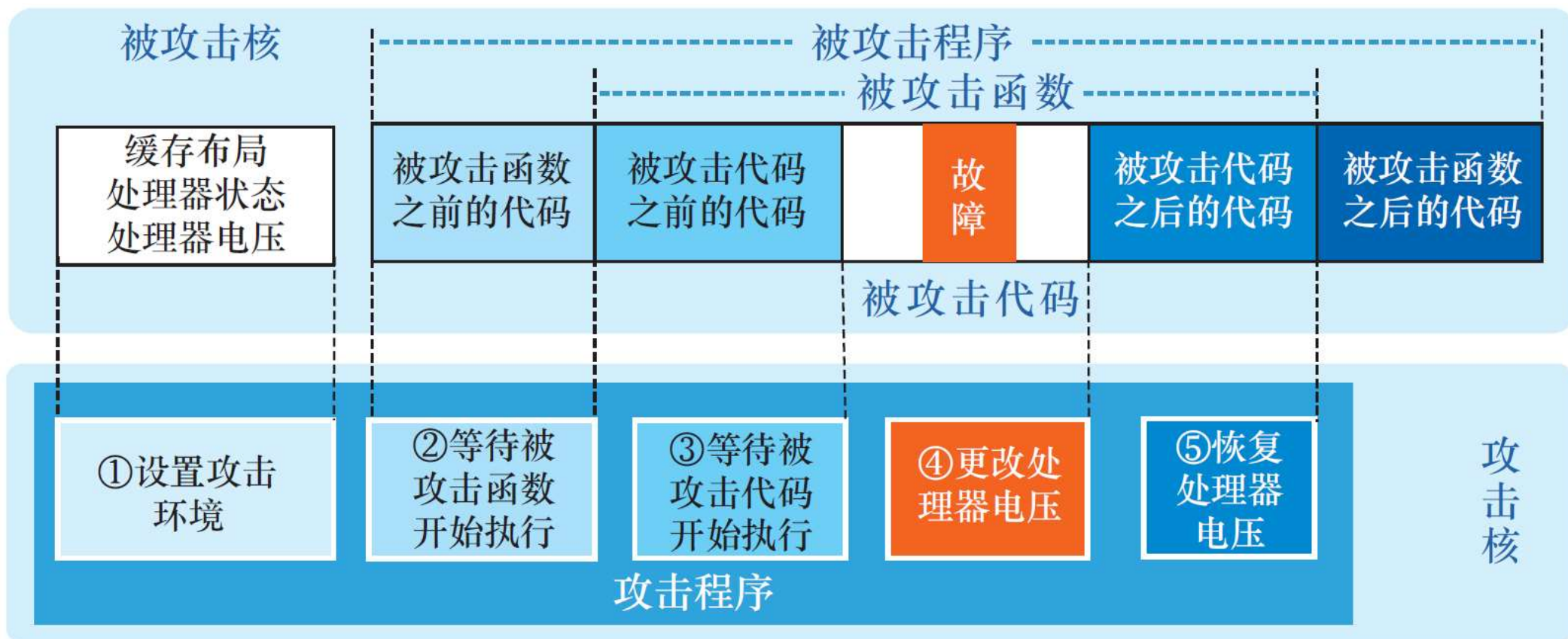
SPECTRE

DEMO分析

1. 訓練CPU的分支預測單元使其在運行代碼時執行特定的預測
2. 分支預測越權訪問敏感數據并將其映射到cache中
3. 利用緩存測信道，通過cache使用情況竊取敏感數據



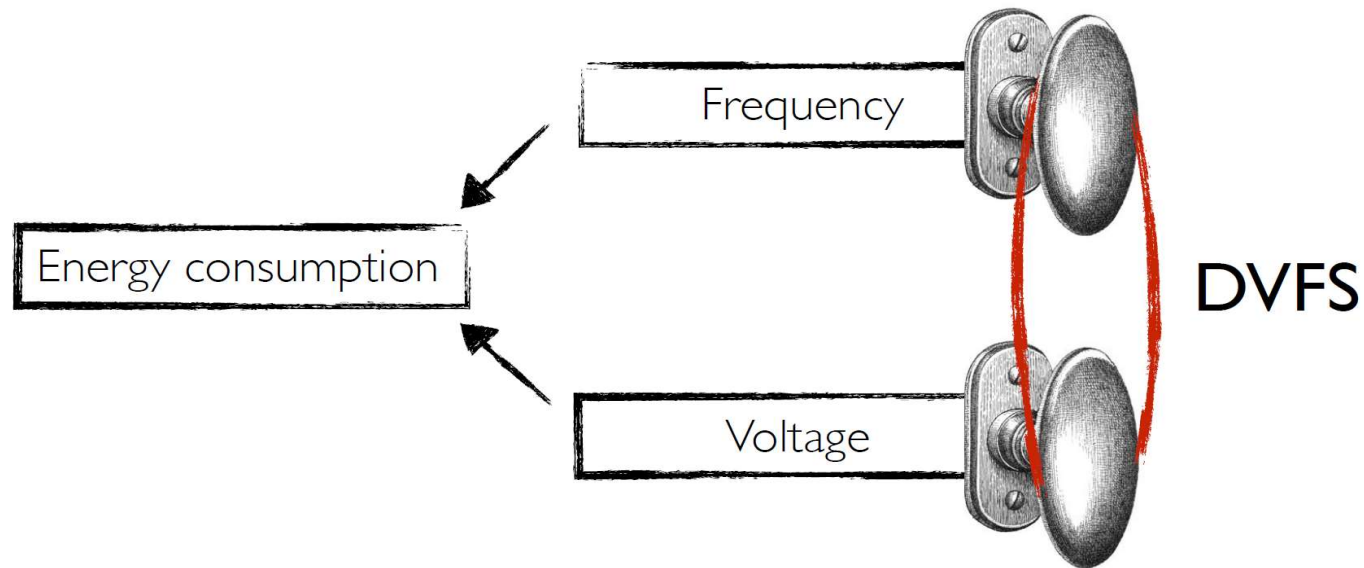
漏洞四： VoltJockey





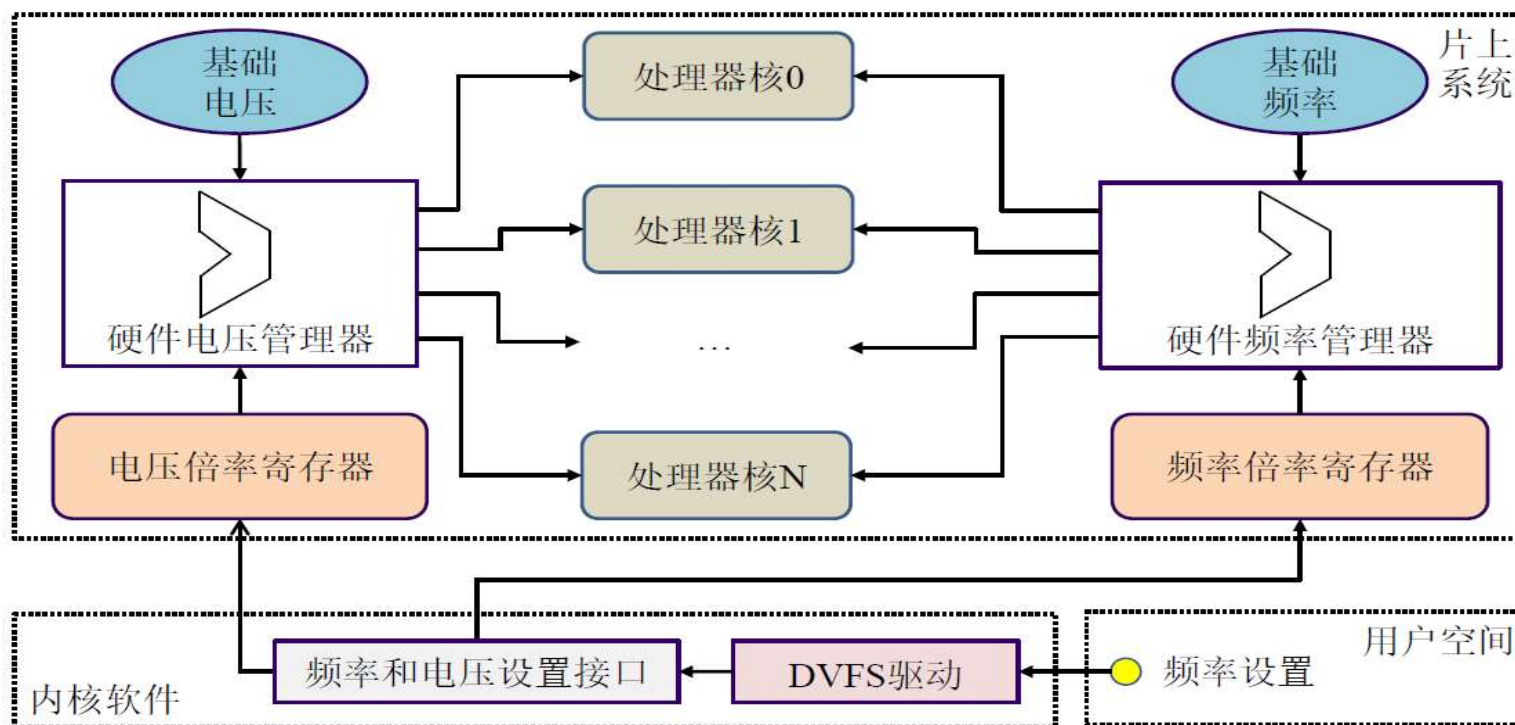
核心動態電源管理技術

- **動態電源管理技術** (Dynamic Voltage and Frequency Scaling, DVFS) 在滿足用戶對性能的需求下根據處理器的負載狀態動態改變電壓和頻率，實現節省能耗的目的，是一種被廣泛應用在現代處理器中的低功耗技術
- 爲了支持DVFS，處理器的硬件頻率和電壓管理器的輸出被設計成是可調的





動態電源管理技術架構



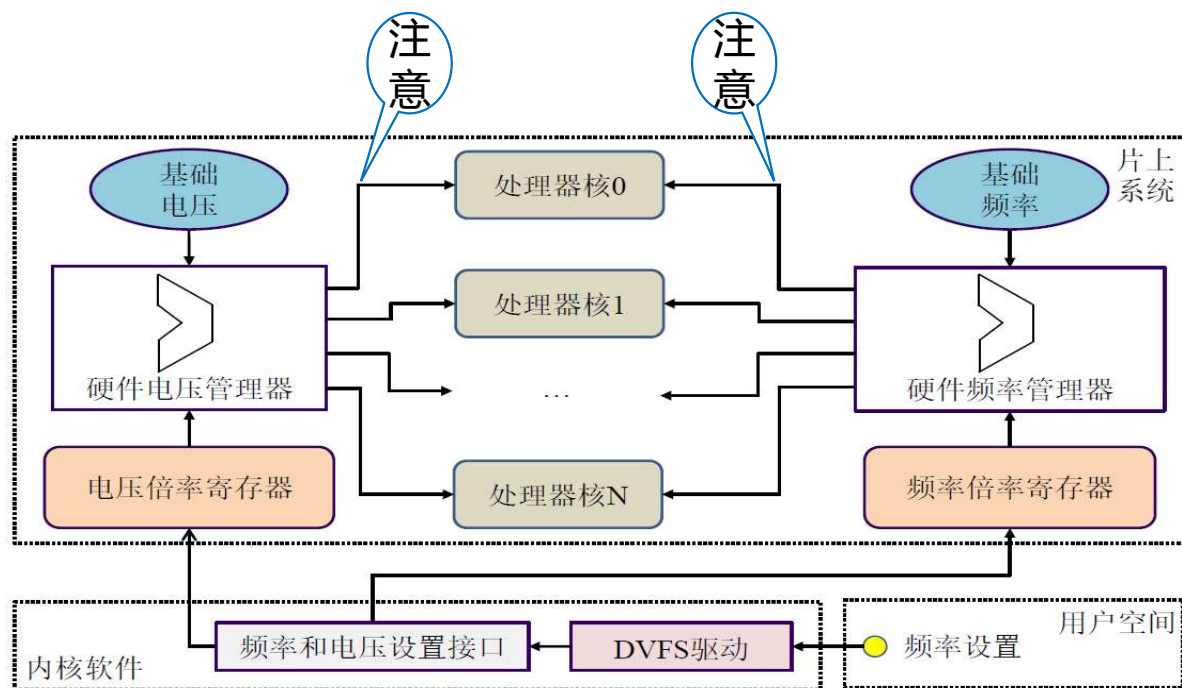
DVFS 基本架構



動態電源管理的漏洞分析

用戶可以設置電壓/頻率管理器參數

ARM不同處理器內核的電壓/頻率可獨立設置



頻率&電壓 時序約束





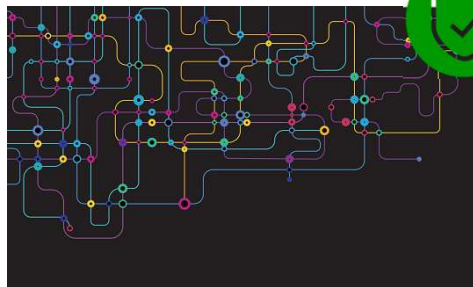
第4節 總結和展望





系統硬件安全

你認為 應該實施哪些策略來保護系統硬件 ？



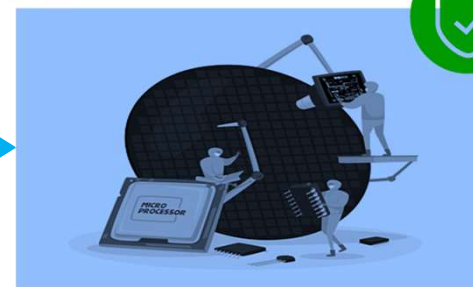
芯片設計

規範編碼
安全版圖設計
可信執行環境



芯片製造

封裝隔離
功能測試
木馬檢測



產品應用

可信啓動
電磁屏蔽
拒絕物理訪問



總結

學習了常用的系統硬件攻擊技術，依附于幾個典型的系統硬件攻擊，對真實的硬件漏洞進行溯源分析，探究了可行的系統硬件防護手段，構建了系統硬件防護體系



第一節 硬件攻擊技術

- 漏洞攻擊
- 硬件木馬
- 硬件故障
- 側信道攻擊

系統設計缺陷、硬件製造缺陷和旁路信息泄露是造成系統硬件漏洞的主要因素



第二節 硬件防護技術

- 木馬檢測技術
- 隔離技術
- 密碼技術
- 側信道防護

系統硬件的產業鏈環境複雜，很難保證設計和生產絕對安全，需要補丁式的硬件防護技術



第三節 典型漏洞分析

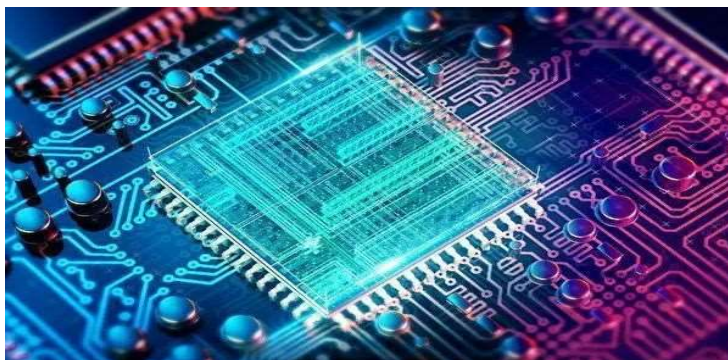
- MOLES
- Meltdown
- Spectre
- VoltJockey

系統硬件漏洞層出不窮且影響廣泛，剖析典型漏洞，探究風險根源，構建硬件防護體系



展望

建設安全無菌，自給自足的系統硬件設計及生產體系



- 突破受制于人的**半導體、光刻機**等卡脖子技術，構建**自主可控**的處理器設計和生產產業鏈
- **人工智能、量子安全**等新技術的發展和應用，為系統硬件的安全設計和生產提供了新的思路



主要内容

沒有硬件安全就沒有網絡空間安全，在萬物互聯的明天，硬件安全問題將面臨更多的威脅

硬件防護技術

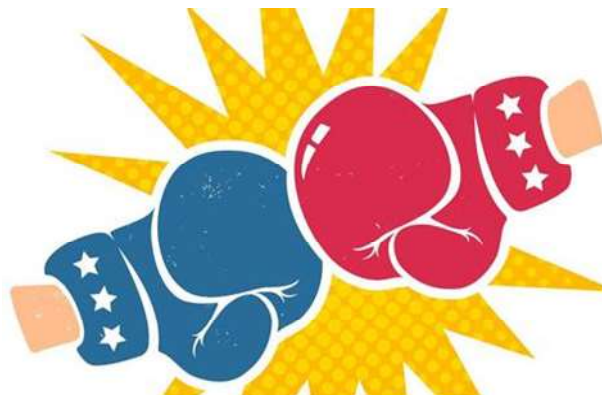


木馬
檢測

隔離
技術

加密
技術

信道
防護



硬件
木馬

漏洞
攻擊

故障
注入

側信
道攻
擊

硬件攻
擊技術

